

Remote Method Invocation (RMI) e Protocolo Requisição-Resposta

Murilo Fragoso e Pedro Erykles

Maio 2026

1 Introdução

Este relatório descreve a reimplementação do sistema de gestão de treinos utilizando Java RMI (Remote Method Invocation), em conformidade com os requisitos do Trabalho 2. O objetivo foi substituir a comunicação por sockets manuais (Trabalho 1) por um protocolo cliente-servidor baseado em invocação remota de métodos, mantendo a estrutura lógica do protocolo requisição-resposta descrita na Seção 5.2 do livro (Coulouris et al.), adaptando-o às restrições acadêmicas e técnicas do enunciado.

2 Arquitetura e Comunicação (RMI vs. Sockets)

A camada de transporte foi delegada inteiramente ao runtime do Java RMI (Unicast RemoteObject, Stub, Skeleton e Registry).

- **Servidor** (*RMI_{Server}*): Registra a implementação remota no `LocateRegistry` na porta 1099 e expõe o serviço via `Naming.rebind()`.
- **Cliente** (*RMI_{Client}*): Localiza o objeto remoto via `Naming.lookup()` e recebe um proxy (stub) que intercepta as chamadas e as encaminha transparentemente pela rede.

Com isso eliminamos a complexidade de gerenciamento de `java.net.Socket`, tratamento de bytes brutos, reconexões e serialização manual de cabeçalhos TCP, focando apenas na lógica de negócio e no protocolo de aplicação.

3 Implementação do Protocolo Requisição-Resposta

O livro descreve o protocolo com três métodos: `doOperation`, `getRequest` e `sendReply`. Em implementações puras com sockets, `getRequest` lê bytes da porta e `sendReply` escreve no socket do cliente. **No contexto RMI, a camada de transporte já é gerenciada pelos stubs/skeletons.** Portanto, para atender ao que foi proposto no trabalho, o protocolo foi adaptado para operar no nível de aplicação (utilizando JSON).

Método do Livro	Implementação	Função
<code>doOperation(RemoteObjectRef o, int methodId, byte[] arguments)</code>	<code>doOperation(String objectRef, String methodId, byte[] arguments)</code>	Ponto de entrada remoto. Recebe <code>byte[]</code> JSON, roteia para a lógica de negócio e retorna <code>byte[]</code> JSON.
<code>getRequest()</code>	<code>getRequest(byte[] arguments)</code>	Desserializa o payload de entrada usando o <code>ObjectMapper</code> configurado. Equivale à “leitura da requisição” no nível de aplicação.
<code>sendReply(byte[] reply, InetAddress clientHost, int clientPort)</code>	<code>sendReply(JsonNode response)</code>	Serializa a resposta para <code>byte[]</code> JSON. Equivale ao “envio da resposta” no nível de aplicação. O endereçamento IP/Porta é gerenciado implicitamente pelo RMI.

4 Representação Externa de Dados (JSON e Jackson)

Como é permitido utilizar JSON como formato para representação externa de dados, a serialização foi feita utilizando a biblioteca Jackson, que já havíamos utilizada no projeto.

4.1 Configuração Unificada do ObjectMapper

Para evitar inconsistências de serialização, o mesmo `ObjectMapper` utilizado nos `IOStreams` do trabalho 1 foi reaproveitado no protocolo RMI:

```
new ObjectMapper()
    .registerModule(new JavaTimeModule())
    .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
    .configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false)
    .configure(MapperFeature.ACCEPT_CASE_INSENSITIVE_ENUMS, true);
```

- `JavaTimeModule`: Serializa `LocalDate` e `DayOfWeek` corretamente.
- `FAIL_ON_UNKNOWN_PROPERTIES = false`: Evita falhas se campos futuros forem adicionados ao JSON.
- `ACCEPT_CASE_INSENSITIVE_ENUMS`: Permite que enums como `INICIANTE`, `iniciante` ou `Iniciante` sejam desserializados sem erro.

4.2 Empacotamento da Mensagem

A mensagem trafega como `byte[]` contendo um objeto JSON no formato:

```

{
    "aluno": {
        "matricula": "ALU-123",
        "nome": "João",
        "nivelExperiencia": "INICIANTE"
    },
    "dia": "MONDAY",
    "treino": { "grupoMuscular": "Peito", "exercicios": [] }
}

```

O `doOperation` extrai `methodId` do contexto da chamada RMI (via roteamento na implementação) e usa os campos JSON como argumentos de negócio, satisfazendo a exigência de representação externa.

5 Passagem por Referência vs. Passagem por Valor

Como o trabalho exige ambos os mecanismos, a implementação foi feita de forma que esteja claramente separados:

Mecanismo	Como foi implementado	Justificativa
Passagem por Referência	O cliente recebe um Stub RMI de <code>IGestaoTreino</code> . As chamadas <code>stub.criarTreino(...)</code> são roteadas para o objeto real no servidor.	Objetos remotos são acessados via proxy. O estado é mantido no servidor (<code>Map bancoAlunos</code>).
Passagem por Valor	<code>Aluno</code> , <code>TreinoDiario</code> , <code>DayOfWeek</code> e respostas são convertidos em <code>byte[]</code> JSON antes do transporte. O servidor desserializa uma cópia local, processa e retorna outro <code>byte[]</code> .	Garante que dados de negócio trafeguem com representação externa independente de plataforma, conforme exigido. O RMI cuida do empacotamento transparente.

6 Conclusão

A comunicação via RMI substitui os sockets manuais, mantendo a robustez e a transparência de localização. O protocolo requisição-resposta foi preservado no nível de aplicação, com `getRequest` e `sendReply` atuando sobre a representação externa de dados (JSON).

Além disso, a utilização da biblioteca Jackson com configuração unificada garante compatibilidade total com os `InputStream/OutputStream` já desenvolvidos, e a estrutura de entidades, agregações, extensões e métodos remotos atende a todos os mínimos exigidos.

7 Referências

1. Coulouris, G., Dollimore, J., & Kindberg, T. Sistemas Distribuídos: Conceitos e Projeto. 5^a ed. Bookman. (Seção 5.2 – Protocolo Requisição-Resposta)
2. Oracle Java Documentation. Remote Method Invocation (RMI). Disponível em: <https://docs.oracle.com/en/java/javase/26/docs/api/java.rmi/java/rmi/package-summary.html>
3. Jackson Documentation. Data Binding, JSR310 Module. Disponível em: <https://github.com/FasterXML/jackson>